



Embedded Systems

Microcontroller, Microprocessor, Embedded Linux

Prateek Rajput | Embedded Linux Developer | Klug Avalon



OUR Plan

- 
- 1- Industry growth and careers
 - 2- Embedded fundamentals
 - 3- Why MCU vs MPU
 - 4- Embedded Linux stack
 - 5- Break
 - 6- Communication protocols
 - 7- Code Structure
 - 8- Roadmap and Q&A



1. Industry Growth

How embedded products and careers grow



Where embedded systems live

- 
- Automotive - ECUs, ADAS, EV battery management
 - Industrial - PLCs, robotics, smart factories
 - Consumer - phones, wearables, smart TVs, home appliances
 - Medical, energy and telecom - monitors, smart meters, routers
 - Almost every modern device now runs software on a small computer



How the embedded industry is structured

- 
- Silicon vendors design chips (ST, NXP, TI, Microchip, Qualcomm)
 - Board and module makers build SoMs and dev boards
 - Software and engineering companies provide BSP, drivers, middleware and apps
 - OEMs integrate everything into a finished product
 - End users get updates over the air, so software work never stops



Why the field keeps growing

- 
- IoT and connected devices need firmware and cloud links
 - Electric vehicles and software-defined vehicles
 - Edge AI running models on small devices
 - Industrial automation and Industry 4.0
 - Government focus on electronics and semiconductor manufacturing

- 
- 
- Trainee > Embedded Engineer > Senior > Tech Lead / Architect > Manager
 - Core - C, C++, data structures, digital electronics
 - Platform - RTOS, Linux kernel and drivers, Yocto / Buildroot, U-Boot
 - Interfaces - UART, I2C, SPI, CAN, USB, Ethernet
 - Tools - GitHub, oscilloscope, logic analyzer, Python scripting



Life of a product

- 
- Idea and requirements
 - Hardware design and board bring-up
 - Bootloader, BSP and drivers
 - Application and middleware
 - Testing, certification and production
 - Field updates (OTA) and long-term maintenance



2. Fundamentals

What an embedded system really is

What is an embedded system?

- A computer built into a product to do one dedicated job
- Resource constrained - limited CPU, memory and power
- Often real-time - results must arrive within a deadline
- Building blocks - processor + memory + I/O + peripherals + power
- Examples - washing machine, smart meter, car ECU, router



Types of embedded systems

- 
- By performance - small (8/16-bit), medium (32-bit), sophisticated (SoC with OS)
 - By timing - hard real-time, soft real-time, non-real-time
 - By software - bare-metal, RTOS-based, Linux-based
 - Choosing the type early drives cost, power and team skills



3. Microcontroller vs Microprocessor

The most asked question in embedded interviews

Microcontroller (MCU)

- CPU + RAM + Flash + peripherals integrated on ONE chip
- Peripherals built in - GPIO, ADC, timers, UART, I2C, SPI
- Runs bare-metal code or an RTOS (boots in milliseconds)
- Low power and low cost (ideal for control tasks)
- Examples - STM32, AVR, PIC, ESP32

Microprocessor (MPU)

- CPU core(s), RAM, storage and I/O sit outside the chip
- Needs external DDR memory, eMMC/SD card and power management
- Runs a full OS such as Linux or Android with an MMU
- Higher speed and capability, higher power, cost and board complexity
- Examples - NXP i.MX, TI Sitara, Raspberry Pi SoC, Intel Atom

Side-by-side comparison

Feature	Microcontroller	Microprocessor
Integration	CPU + memory + peripherals on one chip	CPU memory and I/O external
Clock speed	Roughly MHz to a few hundred MHz	Hundreds of MHz to GHz
Memory	KB to a few MB (on-chip)	Hundreds of MB to GB (external DDR)
Operating system	Bare-metal or RTOS	Linux, Android, Windows
Power use	Very low, can sleep for years on battery	Higher needs power management
Real-time response	Excellent, deterministic	Harder, needs RT patches
Boot time	Instant (ms)	Seconds (bootloader + kernel)
Typical use	Sensors, motor control, appliances	Gateways, HMI, cameras, infotainment

How to choose (MCU or MPU)?

- Simple control, tight deadlines, battery power - choose an MCU
- Needs networking stack, GUI, camera, file system - choose an MPU
- Need both? Use an MPU plus a companion MCU, or a hybrid chip (STM32MP1, i.MX8 with Cortex-M)
- Think cost per unit, power budget, security and available skills



4. Embedded Linux

From power-on to your application

Embedded Linux boot flow

- 1. ROM code inside the SoC loads the first bootloader
- 2. Bootloader (U-Boot) initialises DDR and loads kernel + device tree
- 3. Linux kernel starts drivers and mounts the root filesystem
- 4. init (systemd / BusyBox) starts services
- 5. Your application runs in user space



Parts of an embedded Linux system

- 
- Toolchain - cross-compiler that builds code on a PC for the target
 - Bootloader - U-Boot
 - Kernel + device tree - describes the hardware to the kernel
 - Root filesystem - libraries, tools, init, applications
 - Build systems - Yocto Project and Buildroot create all of the above



Kernel space vs user space

- 
- Kernel space - drivers, scheduling, memory management, full hardware access
 - User space - applications, protected from crashing the whole system
 - Talk to drivers through /dev nodes, sysfs and system calls
 - Rule - keep drivers small, keep logic in applications



5. Communication Protocols

How chips and devices talk - UART, I2C, SPI, I2S, USB, CAN and CAN FD

Protocols at a glance

Protocol	Wires	Clock	Typical speed	Topology	Best for
UART	2 (TX, RX)	Asynchronous	115200 baud common, up to Mbps	Point-to-point	Debug console, modules
I2C	2 (SDA, SCL)	Synchronous	100 kbps - 3.4 Mbps	Multi-drop, addressed	Sensors, EEPROM, RTC
SPI	4+ (SCLK, MOSI, MISO, CS)	Synchronous	1 - 50+ MHz	Master + slaves	Flash, displays, ADC
I2S	3-4 (BCLK, LRCLK, SD, MCLK)	Synchronous	44.1 / 48 kHz audio	Point-to-point	Audio codecs, mics
USB	4 (USB 2.0)	Inside the data	1.5 Mbps - 5+ Gbps	Host + tiered hubs	Peripherals, storage
CAN	2 (CAN_H, CAN_L)	Inside the data	Up to 1 Mbps	Multi-master bus	Vehicles, industry
CAN FD	2 (CAN_H, CAN_L)	Inside the data	Data phase 2 - 5 Mbps	Multi-master bus	Modern ECUs, ADAS

UART | serial port

Wires

TX, RX, GND

Speed

9600 - 115200 baud+

Clock

None (asynchronous)

Typical use

Debug console, GPS, GSM

- Both ends must agree on baud rate, data bits, parity and stop bits (e.g. 115200 8N1)
- Frame: start bit, 5-9 data bits, optional parity bit, 1-2 stop bits
- Cross the lines - TX of one device goes to RX of the other
- Full duplex and point-to-point RS-232 and RS-485 are electrical standards on top
- Linux - /dev/ttyS0 or /dev/ttyUSB0 the first thing you connect on a new board

I2C | two-wire bus

Wires

SDA, SCL (+ GND)

Speed

100 kbps - 3.4 Mbps

Topology

Multi-master, multi-slave

Typical use

Sensors, EEPROM, RTC

- Open-drain lines need pull-up resistors (commonly 4.7k ohm)
- Each slave has a 7-bit address; the master starts every transfer
- Sequence - START, address + R/W bit, ACK, data bytes with ACK, STOP
- Modes - Standard 100 kbps, Fast 400 kbps, Fast-plus 1 Mbps, High-speed 3.4 Mbps
- Linux - /dev/i2c-N scan with i2c detect, read and write with i2c get and i2c set

SPI | fast synchronous link

Wires

SCLK, MOSI, MISO, CS

Speed

Often 1 - 50+ MHz

Clock

Synchronous, full duplex

Typical use

Flash, display, ADC, SD

- The master drives the clock; one chip-select line per slave device
- Modes 0-3 are set by clock polarity (CPOL) and phase (CPHA) both sides must match
- Faster than I2C with no addressing overhead, but needs more pins
- No built-in ACK or error checking software must verify the data
- Linux - /dev/spidevX.Y or a proper kernel driver using the SPI subsystem

I2S | digital audio

Wires

BCLK, LRCLK, SD (+ MCLK)

Data

Digital audio (PCM)

Rates

44.1 / 48 kHz, 16-32 bit

Typical use

Codec, DAC, MEMS mic

- BCLK (bit clock) times each bit LRCLK (word select) marks left or right channel
- SD carries the audio samples, most significant bit first
- One device is the clock master (often the SoC) the other follows
- Optional MCLK gives the codec a clean master clock
- Linux - the ALSA ASoC framework ties the SoC I2S port, codec and machine driver

USB | universal serial bus

Wires

VBUS, D+, D-, GND (USB 2.0)

Speed

1.5 Mbps to 5+ Gbps

Topology

Host-controlled tiered star

Typical use

Storage, HID, serial, camera

- The host starts everything; devices only answer when polled - up to 127 devices per host
- Speeds - Low 1.5 Mbps, Full 12 Mbps, High 480 Mbps, SuperSpeed 5 Gbps and above
- Enumeration - device attaches, host reads descriptors, loads the matching class driver
- Transfer types - control, bulk, interrupt, isochronous
- Classes - HID, CDC (serial), MSC (storage) Linux boards can be host or gadget

CAN | controller area network

Wires

**CAN_H, CAN_L
(differential)**

Speed

Up to 1 Mbps

Topology

Multi-master bus

Typical use

**Vehicles, factory
machines**

- Message based - an ID (11-bit standard or 29-bit extended) names the data, not the device
- The lowest ID wins arbitration without losing the message, so priority is built in
- Up to 8 data bytes per frame, with CRC, ACK and automatic error handling
- Needs 120 ohm termination resistors at both ends of the bus
- Linux - SocketCAN gives a can0 interface; use candump, cansend and ip link

CAN FD | flexible data-rate

Payload

Up to 64 bytes

Data speed

Commonly 2 - 5 Mbps

Arbitration

Same as classic CAN

Typical use

Modern ECUs, ADAS, OTA

- The data phase switches to a faster bit rate after arbitration
- Payload grows from 8 to 64 bytes, so far fewer frames carry the same data
- A stronger CRC protects the larger frames
- Classic CAN nodes cannot share a bus with FD frames plan the network upgrade
- Linux - ip link set can0 type can bitrate 500000 dbitrade 2000000 fd on

Choosing a protocol

- Debug or a simple module link - UART
- Several slow sensors on two wires - I2C
- High-speed chip link (flash, display, ADC) - SPI
- Digital audio - I2S
- PC-style devices and storage - USB
- Noisy, long, multi-node networks (vehicles, factories) CAN, or CAN FD for more data



6. Code Structure

How firmware and Linux code are organised

Bare-metal MCU startup to main()

```
// 1. Reset handler (startup.s) sets stack,
//    copies .data, zeroes .bss, calls main()
#include "hal_gpio.h"
volatile uint8_t tick;
void SysTick_Handler(void){ tick = 1; }

int main(void){
    HAL_Init();           // clocks and pins
    LED_Init();
    while(1){             // super-loop
        if(tick){ tick = 0; LED_Toggle(); }
    }
}
```

- Startup code and vector table run before main()
- Linker script maps Flash and RAM sections
- Super - loop plus interrupts is the classic pattern
- volatile - variable changes outside normal flow
- Keep ISRs short set a flag, work in main

Layered architecture

- Application layer - product logic and state machines
- Middleware - protocol stacks, file system, RTOS
- HAL / driver layer - talks to registers, hides chip details
- Hardware - MCU or SoC and peripherals
- Why layers? Portability, testing, teamwork, easy chip changes

Typical project layout

```
project/  
|-- src/          main.c, app logic  
|-- drivers/      uart.c, spi.c, i2c.c  
|-- hal/          hardware abstraction  
|-- include/      *.h public headers  
|-- startup.s     vector table  
|-- linker.ld     memory map  
|-- Makefile      build rules  
|-- README.md     how to build and flash
```

- One module = one .c and one .h file
- Headers declare, source files define
- Hide internals with static functions
- Separate hardware code from application logic

Linux kernel module skeleton

```
// hello.c
#include <linux/module.h>

static int __init hello_init(void){
    pr_info("hello: loaded\n");
    return 0;
}
static void __exit hello_exit(void){
    pr_info("hello: bye\n");
}
module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

- init runs on insmod, exit runs on rmmod
- pr_info writes to the kernel log (dmesg)
- Real drivers add probe(), device tree match and file operations
- Build with a kernel Makefile, test on target hardware



Good coding practices

- 
- Clear names, small functions, one job per function
 - Comment why, not what
 - Check return values and handle errors
 - No dynamic memory in tight loops; watch stack size
 - Follow a standard (MISRA C, Linux kernel style)
 - Use Git, code review and test early on hardware



7. Hands-on and Roadmap

Turn knowledge into skills

Hands-on exercises

- Exercise 1: blink an LED on an MPU board (GPIO + timer)
- Exercise 2: print sensor data over UART
- Exercise 3: boot a Linux board and read dmesg
- Exercise 4: build and load the hello kernel module

Student roadmap

- Months 1-2 - strong C, digital electronics, Git
- Months 3-4 - STM32 or ESP32 projects, UART/I2C/SPI
- Months 5-6 - Raspberry Pi / BeagleBone with Linux, Yocto basics
- Build 2-3 portfolio projects and publish them on GitHub



Q&A

THANKS

Please ask Questions ?

Prateek Rajput, Klug Avalon



Learn • Build • Create with Electronics



How To Electronics

@HowtoElectronics • 103k subscribers • 343 videos

Welcome to How To Electronics – a place to learn, build, and create with electronics. ...more

how2electronics.com and 3 more links

Subscribed

Community

Home

Videos

Shorts

Playlists

Posts

For you



DIY Portable ECG Monitor

DIY ECG Monitor with ESP32 + ADS1292R | ECG, BPM & Respiration

10:15



Measuring Acceleration...



IoT Air Quality Monitor

ESP32 + PMS7003 + BME680

10:53



Patient Health Monitor

ESP32 Patient Health Monitor with L Dashboard | BPM, SpO₂, ECG & Temp

10:53



All About Electronics



Just Do Electronics

@JustDoElectronics • 22.9k subscribers • 489 videos

Welcome to Just Do Electronics ...more

justdoelectronics.com and 3 more links

Subscribed

Community

Home

Videos

Shorts

Playlists

Posts

Latest Popular Oldest



CONTROL THIS

Control a Stepper Motor via Web Server | ESP32 & DRV8825 Tutorial

4:53



REMOTE CONTROL

I Made a Wireless SOS Location Tracker with Arduino & RF

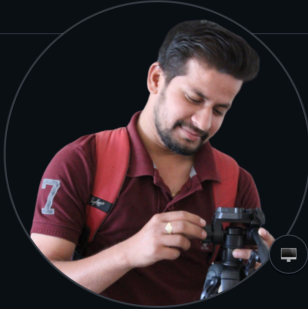
4:34



BUILD A SMART TROLLEY WITH GROW-GM861S SCANNER!

How to Build an IoT Smart Shopping Trolley | ESP32, Barcode Scanner & Web Dashboard

4:11



Overview

Repositories 97

Projects

Packages

Stars 4

Find a repository... Type Language Sort

MAX7219_Dot_Matrix_Projects

Public

C++ 1 Updated 3 weeks ago

Raspberry-Pi-4-With-Logitech-C270-USB-Webcam

Public

HTML Updated on Aug 24

CAN-Communication

Public

C++ Updated on Jul 29

PrateekSinghRajput

Public

Config files for my GitHub profile.

config github-config

1 Updated on Jul 29

3.5-inch-ILI9486-TFT-Touch-Shield-LCD-Module

Public

C++ Updated on Jul 28

Prateek Singh

PrateekSinghRajput

Follow

Embedded Developer || C, C++, Python || MTech Student

275 followers • 0 following

Pune , India

https://www.justdoelectronics.com

@JustDoElectronics

@HowtoElectronics

justdoelectronics

in/prateek-singh-rajpud-aa235b245